

CERTIA v3 — Prompt de inicio para Claude Code

Contexto

Sos el desarrollador principal de **CERTIA v3**, un sistema de gestión de certificados Halal para el **Centro Islámico del Uruguay**. El documento adjunto (`CERTIA_v3_Manual_de_Flujos.docx`) es la especificación completa del sistema. Leelo íntegramente antes de escribir una sola línea de código.

Stack tecnológico definido

Capa	Tecnología
Backend	Node.js + Express
Frontend	React + Vite + Tailwind CSS
Base de datos	PostgreSQL
ORM	Prisma
Autenticación	JWT + bcrypt + Email OTP (2FA admin)
Generación PDF	Puppeteer
Firma digital	firma.gub.uy (manual) + validación con pdf-lib + node-forge
Tiempo real	Socket.IO
Emails	Nodemailer
IA plantillas	Claude API (claude-sonnet-4-20250514) con visión
Almacenamiento	Cloudinary
Export Excel	ExcelJS
PWA	Vite PWA Plugin + Workbox

Lo que necesito que hagas

FASE 1 — Estructura y base del proyecto

Creá la estructura completa del monorepo:

```
certia/
├── backend/
│   ├── src/
│   │   ├── controllers/
│   │   ├── routes/
│   │   ├── middleware/
│   │   ├── services/
│   │   ├── jobs/           ← cron jobs (vencimiento, OTP cleanup)
│   │   ├── sockets/       ← Socket.IO events
│   │   ├── utils/
│   │   └── app.js
│   ├── prisma/
│   │   └── schema.prisma  ← esquema completo según el manual
│   ├── .env.example
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   │   ├── admin/
│   │   │   └── cliente/
│   │   ├── components/
│   │   ├── hooks/         ← useSocket, useAuth, useNotifications
│   │   ├── stores/        ← Zustand
│   │   └── main.jsx
│   ├── public/
│   │   └── manifest.json  ← PWA
│   ├── vite.config.js
│   └── package.json
└── README.md
```

FASE 2 — Base de datos completa

Generá el `schema.prisma` con **todos** los modelos del manual:

- Admin con campo `activo`

- `OtpCode` con `codigoHash`, `expiraEn`, `usado`, `intentosFallidos`
- `Cliente` con `nombreEmpresa`, `creadoPor`
- `TipoCertificado` con `camposFormulario Json`, `plantillaHtml`, `variablesMapeadas Json`, `versionPlantilla`
- `VersionPlantilla`
- `ClienteTipoCert` (pivot)
- `Solicitud` con `datosFormulario Json`, `estadoActual`, `pdfBorradorUrl`, `pdfSinFirmaUrl`, `pdfFirmadoUrl`, `qrToken`, `fechaVencimiento`, `renovacionDeId` (self-relation opcional)
- `HistorialEstado` con `motivoInterno` y `motivoCliente` separados
- `Comentario` con enum `tipo` (INTERNO | VISIBLE)
- `Adjunto`
- `Auditoria` con `datosAnteriores Json`, `datosNuevos Json`
- `Notificacion` con `estadoEnvio`, `intentos`, `error`
- `Webhook` con `eventos Json`, `secret`
- `WebhookEntrega` con `httpStatus`, `exito`, `intentos`
- `Configuracion` (clave-valor)

Enum de estados: `PENDIENTE`, `EN_REVISION`, `OBSERVADO`, `PAGO_VALIDADO`, `EN_ELABORACION`, `REVISION_PDF`, `PENDIENTE_FIRMA`, `FINALIZADO`, `RECHAZADO`, `VENCIDO`

FASE 3 — Backend completo

Implementá en orden:

1. Autenticación (/api/auth)

- `POST /login` → valida credenciales → si admin genera OTP y lo envía por email → responde `{otpRequerido: true}`
- `POST /verify-otp` → valida OTP → emite JWT access + refresh token (cookie `HttpOnly`)
- `POST /refresh` → renueva access token
- `POST /logout`

- Cliente: `POST /login` → directo a JWT (sin OTP)

2. Middleware

- `authMiddleware` → valida JWT, adjunta `req.user`
- `roleMiddleware('admin'|'cliente')` → verifica rol
- `auditMiddleware` → registra acciones en tabla Auditoria
- Rate limiting con `express-rate-limit`
- Helmet

3. Solicitudes (`/api/solicitudes`)

- CRUD completo con cambios de estado
- En cada cambio de estado: actualizar `estadoActual`, insertar en `HistorialEstado`, disparar email (Nodemailer), emitir evento Socket.IO a la sala del destinatario
- Al pasar a `EN_ELABORACION`: trigger automático de generación de PDF borrador (Puppeteer) → guardar en Cloudinary → pasar a `REVISION_PDF`
- Al pasar a `PENDIENTE_FIRMA`: disponibilizar descarga del PDF sin firma
- Upload de PDF firmado: validar con `pdf-lib` + `node-forge` → si válido agregar marca de agua → guardar en Cloudinary → pasar a `FINALIZADO` automáticamente
- Al `FINALIZADO`: generar QR token único, calcular fecha de vencimiento, emitir webhooks configurados

4. Comentarios (`/api/solicitudes/:id/comentarios`)

- `GET` → admin ve todos; cliente ve solo `VISIBLE`
- `POST` → admin crea `INTERNO` o `VISIBLE`; si `VISIBLE` dispara email + WS al cliente

5. Clientes (`/api/clientes`) — CRUD admin

6. Tipos de certificado (`/api/tipos`) — CRUD admin + historial de versiones

7. IA plantillas (`/api/ia/generar-plantilla`)

- Recibe PDF → convierte a imágenes con `pdf-poppler` → envía a Claude API Vision → devuelve `{html, campos, nombre, descripcion}`

8. Webhooks (`/api/webhooks`) — CRUD admin + historial de entregas + retry con backoff

9. **Export Excel** (`/api/export`) — solicitudes, certificados, historial estados, auditoría
10. **Verificación pública** (`GET /verify/:token`) — sin auth
11. **Cron jobs** (node-cron):
 - Diario 08:00 Uruguay: vencimientos + avisos anticipados (30/15/7 días) + cleanup OTPs expirados

FASE 4 — Frontend completo

Portal Admin (`/admin/*`):

- Login con flujo OTP (pantalla de contraseña → pantalla de código)
- Dashboard con métricas y notificaciones tiempo real
- Listado de solicitudes con filtros, búsqueda y export
- Vista de detalle de solicitud:
 - Timeline visual de estados
 - Panel de cambio de estado con campo de motivo
 - Preview PDF en REVISION_PDF (iframe o embed)
 - Botón descarga PDF sin firma en PENDIENTE_FIRMA
 - Upload PDF firmado con validación en tiempo real
 - Sección de comentarios (internos y visibles)
- CRUD clientes con toggle de tipos habilitados
- CRUD tipos de certificado + Monaco Editor para plantilla HTML + wizard IA
- Configuración del sistema (vencimiento, SMTP, Claude API key, webhooks)
- Log de auditoría con filtros
- Export a Excel desde listado

Portal Cliente (`/cliente/*`):

- Login directo (sin OTP)
- Dashboard con solicitudes activas, alertas OBSERVADO, próximos vencimientos
- Nueva solicitud — wizard 3 pasos

- Vista de detalle: timeline, motivo OBSERVADO, comentarios visibles, botón descarga PDF
- Renovación: botón en certificados próximos a vencer → formulario pre-llenado
- Notificaciones en tiempo real (toast + badges)

Shared:

- `useSocket()` hook — conecta Socket.IO, escucha eventos por sala del usuario
 - `useAuth()` hook — maneja JWT, refresh automático
 - `useNotifications()` hook — acumula eventos WS en estado global
 - Zustand store para estado global
 - Tailwind config con paleta CERTIA (verde `#1A6B3C` , dorado `#C8A84B`)
 - PWA: `vite.config.js` con VitePWA plugin, `manifest.json` , service worker con Workbox
-

Reglas de desarrollo

1. **Leé el manual completo antes de empezar.** Cada decisión de diseño está justificada ahí.
2. **Implementá en las fases en orden.** No saltes a frontend sin tener el schema de Prisma validado.
3. **Cada cambio de estado dispara tres cosas simultáneamente:** actualización en DB + email (Nodemailer) + evento WebSocket (Socket.IO). Sin excepción.
4. **Los comentarios INTERNO nunca llegan al cliente** — ni por API, ni por email, ni por WebSocket.
5. **El motivo en OBSERVADO y RECHAZADO es obligatorio.** El endpoint debe retornar 400 si está vacío.
6. **FINALIZADO y RECHAZADO son estados terminales.** El endpoint debe rechazar cualquier intento de cambio.
7. **La validación del PDF firmado no es opcional.** Antes de pasar a FINALIZADO, siempre se ejecuta la validación con pdf-lib + node-forge.
8. **Todos los archivos subidos se renombran con UUID.** Nunca se usa el nombre original

del usuario.

9. **Variables de entorno para todo.** Nunca credenciales en código fuente.
10. **El schema de Prisma es la fuente de verdad.** No escribas SQL crudo.

Variables de entorno (.env.example)

```
# Base de datos
DATABASE_URL=postgresql://user:pass@host:5432/certia

# JWT
JWT_SECRET=
JWT_REFRESH_SECRET=

# Claude API
ANTHROPIC_API_KEY=

# Cloudinary
CLOUDINARY_URL=

# Email (SMTP)
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASS=
EMAIL_FROM=CERTIA <certia@centroislamico.uy>

# Sistema
APP_URL=http://localhost:3000
FRONTEND_URL=http://localhost:5173
CERT_EXPIRY_DAYS=365
NOTIFY_DAYS_BEFORE=30,15,7
RENEWAL_ENABLE_DAYS=60
OTP_EXPIRY_MINUTES=10
```

Primer paso concreto

Comenzá por:

1. Crear la estructura de carpetas completa del monorepo

2. Inicializar `package.json` en backend y frontend con todas las dependencias necesarias
3. Generar el `schema.prisma` completo con todos los modelos y enums
4. Ejecutar `npx prisma generate` y `npx prisma migrate dev --name init`
5. Crear el seed inicial (`prisma/seed.js`) con:
 - Un admin por defecto: `admin@certia.uy / Admin1234!`
 - Los 6 tipos de certificado base (HC01–HC06) como inactivos
 - Las configuraciones por defecto en la tabla `Configuracion`
6. Confirmarme la estructura creada y el output de la migración antes de continuar con el backend

Adjunto el documento `CERTIA_v3_Manual_de_Flujos.docx` como especificación de referencia. Cualquier ambigüedad que no esté resuelta en este prompt, consultala con el manual antes de tomar una decisión.